

HUNDRED DAYS OF CODE

DAY 18

QUESTION:

Write a menu-driven C program using **switch-case** to perform digit-based operations on a positive integer. Define **one separate user-defined function for each menu option**, with *void return type* and *with a simple function call (without call by value/reference)*. Each function should read the required input, process it, and display the result. Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

- Check whether it is a Strong number.
- Check whether it is an Armstrong number.
- Check whether it is a Harshad number.
- Check whether it is a Duck number.

ALGORITHM:

Main Algorithm

1. Start.
2. Display the menu:
 - Strong Number
 - Armstrong Number
 - Harshad Number
 - Duck Number
 - Exit
3. Read the user's choice.
4. Use switch-case to select the required operation.
5. Call the corresponding void function.
6. The selected function reads the positive integer, performs the operation, and displays the result.
7. Repeat the menu until the user selects Exit.
8. Stop.

Algorithm for Strong Number

1. Read a positive integer n.

HUNDRED DAYS OF CODE

DAY 18

2. Store n in temp.
3. Set sum = 0.
4. Extract each digit using $\text{digit} = \text{temp} \% 10$.
5. Find the factorial of the digit.
6. Add the factorial to sum.
7. Remove the last digit using $\text{temp} = \text{temp} / 10$.
8. Repeat until temp becomes 0.
9. If $\text{sum} == n$, the number is a Strong number; otherwise, it is not.
10. Display the result.

Example:

$145 = 1! + 4! + 5! = 1 + 24 + 120 = 145$, so 145 is a Strong number.

Algorithm for Armstrong Number

1. Read a positive integer n.
2. Store n in temp.
3. Count the number of digits, say digits.
4. Set sum = 0.
5. Extract each digit.
6. Calculate $\text{digit}^{\text{digits}}$.
7. Add the result to sum.
8. Remove the last digit.
9. Repeat until temp becomes 0.
10. If $\text{sum} == n$, the number is an Armstrong number; otherwise, it is not.
11. Display the result.

Example:

$153 = 1^3 + 5^3 + 3^3 = 153$, so 153 is an Armstrong number.

Algorithm for Harshad Number

HUNDRED DAYS OF CODE

DAY 18

1. Read a positive integer n .
2. Store n in temp.
3. Set $sum = 0$.
4. Extract each digit from n .
5. Add every digit to sum .
6. If $n \% sum == 0$, the number is a Harshad number.
7. Otherwise, it is not a Harshad number.
8. Display the result.

Example:

$18 \rightarrow 1 + 8 = 9$ and $18 \% 9 = 0$, so 18 is a Harshad number.

Algorithm for Duck Number

1. Read a positive integer n .
2. Store n in temp.
3. Set $flag = 0$.
4. Extract each digit.
5. If any digit is 0, set $flag = 1$.
6. Continue until all digits are checked.
7. If $flag == 1$, the number is a Duck number.
8. Otherwise, it is not a Duck number.
9. Display the result.

Example:

1023 contains 0, so it is a Duck number.

3. Pseudocode

START

REPEAT

HUNDRED DAYS OF CODE

DAY 18

Display menu

1. Strong Number
2. Armstrong Number
3. Harshad Number
4. Duck Number
5. Exit

Read choice

SWITCH(choice)

CASE 1:

CALL strongNumber()

BREAK

CASE 2:

CALL armstrongNumber()

BREAK

CASE 3:

CALL harshadNumber()

BREAK

CASE 4:

CALL duckNumber()

BREAK

HUNDRED DAYS OF CODE

DAY 18

CASE 5:

Display "Exiting program"

STOP

DEFAULT:

Display "Invalid choice"

END SWITCH

UNTIL choice = 5

END

Pseudocode for strongNumber()

FUNCTION strongNumber()

Read n

temp = n

sum = 0

WHILE temp > 0

digit = temp MOD 10

fact = 1

FOR i = 1 TO digit

fact = fact * i

END FOR

HUNDRED DAYS OF CODE

DAY 18

```
sum = sum + fact
```

```
temp = temp DIV 10
```

```
END WHILE
```

```
IF sum = n
```

```
    Display "Strong number"
```

```
ELSE
```

```
    Display "Not a Strong number"
```

```
END IF
```

```
END FUNCTION
```

Pseudocode for armstrongNumber()

```
FUNCTION armstrongNumber()
```

```
    Read n
```

```
    temp = n
```

```
    digits = 0
```

```
    WHILE temp > 0
```

```
        digits = digits + 1
```

```
        temp = temp DIV 10
```

```
    END WHILE
```

```
    temp = n
```

```
    sum = 0
```

HUNDRED DAYS OF CODE

DAY 18

WHILE temp > 0

 digit = temp MOD 10

 sum = sum + digit RAISED TO digits

 temp = temp DIV 10

END WHILE

IF sum = n

 Display "Armstrong number"

ELSE

 Display "Not an Armstrong number"

END IF

END FUNCTION

Pseudocode for harshadNumber()

FUNCTION harshadNumber()

 Read n

 temp = n

 sum = 0

 WHILE temp > 0

 digit = temp MOD 10

 sum = sum + digit

 temp = temp DIV 10

 END WHILE

 IF n MOD sum = 0

HUNDRED DAYS OF CODE

DAY 18

Display "Harshad number"

ELSE

Display "Not a Harshad number"

END IF

END FUNCTION

Pseudocode for duckNumber()

FUNCTION duckNumber()

Read n

temp = n

flag = 0

WHILE temp > 0

digit = temp MOD 10

IF digit = 0

flag = 1

END IF

temp = temp DIV 10

END WHILE

IF flag = 1

Display "Duck number"

ELSE

Display "Not a Duck number"

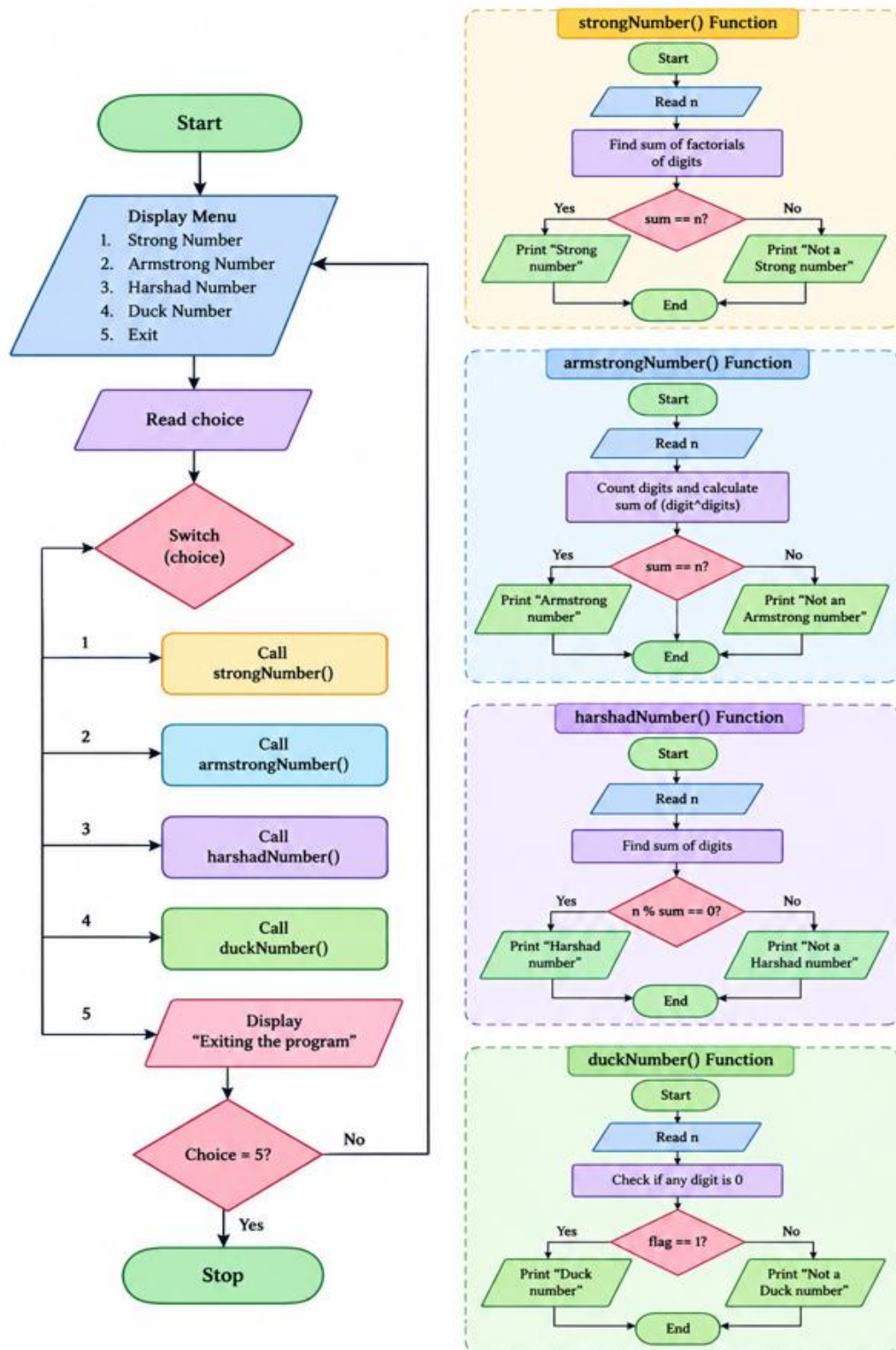
HUNDRED DAYS OF CODE

DAY 18

END IF

END FUNCTION

4. FLOWCHART:



HUNDRED DAYS OF CODE

DAY 18

5. C Program

```
#include <stdio.h>
```

```
/* Function to check Strong Number */
```

```
void strongNumber()
```

```
{
```

```
    int n, temp, digit, fact, sum = 0, i;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    temp = n;
```

```
    while (temp > 0)
```

```
    {
```

```
        digit = temp % 10;
```

```
        fact = 1;
```

```
        for (i = 1; i <= digit; i++)
```

```
        {
```

```
            fact = fact * i;
```

```
        }
```

```
        sum = sum + fact;
```

```
        temp = temp / 10;
```

```
    }
```

HUNDRED DAYS OF CODE

DAY 18

```
if (sum == n)

    printf("%d is a Strong number.\n", n);

else

    printf("%d is not a Strong number.\n", n);

}
```

```
/* Function to check Armstrong Number */
```

```
void armstrongNumber()
```

```
{
```

```
    int n, temp, digit, digits = 0, sum = 0;
```

```
    int i, power;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

```
    temp = n;
```

```
    /* Count number of digits */
```

```
    while (temp > 0)
```

```
    {
```

```
        digits++;
```

```
        temp = temp / 10;
```

```
    }
```

```
    temp = n;
```

HUNDRED DAYS OF CODE

DAY 18

```
/* Calculate sum of powers of digits */
while (temp > 0)
{
    digit = temp % 10;
    power = 1;

    for (i = 1; i <= digits; i++)
    {
        power = power * digit;
    }

    sum = sum + power;
    temp = temp / 10;
}

if (sum == n)
    printf("%d is an Armstrong number.\n", n);
else
    printf("%d is not an Armstrong number.\n", n);
}

/* Function to check Harshad Number */
void harshadNumber()
{
    int n, temp, digit, sum = 0;
```

HUNDRED DAYS OF CODE

DAY 18

```
printf("Enter a positive integer: ");
```

```
scanf("%d", &n);
```

```
temp = n;
```

```
while (temp > 0)
```

```
{
```

```
    digit = temp % 10;
```

```
    sum = sum + digit;
```

```
    temp = temp / 10;
```

```
}
```

```
if (n % sum == 0)
```

```
    printf("%d is a Harshad number.\n", n);
```

```
else
```

```
    printf("%d is not a Harshad number.\n", n);
```

```
}
```

```
/* Function to check Duck Number */
```

```
void duckNumber()
```

```
{
```

```
    int n, temp, digit, flag = 0;
```

```
    printf("Enter a positive integer: ");
```

```
    scanf("%d", &n);
```

HUNDRED DAYS OF CODE

DAY 18

```
temp = n;

while (temp > 0)
{
    digit = temp % 10;

    if (digit == 0)
    {
        flag = 1;
        break;
    }

    temp = temp / 10;
}

if (flag == 1)
    printf("%d is a Duck number.\n", n);
else
    printf("%d is not a Duck number.\n", n);
}

/* Main function */
int main()
{
    int choice;
```

HUNDRED DAYS OF CODE

DAY 18

```
do
{
    printf("\n===== MENU =====\n");
    printf("1. Check Strong Number\n");
    printf("2. Check Armstrong Number\n");
    printf("3. Check Harshad Number\n");
    printf("4. Check Duck Number\n");
    printf("5. Exit\n");
    printf("===== \n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            strongNumber();
            break;

        case 2:
            armstrongNumber();
            break;

        case 3:
            harshadNumber();
            break;
```

HUNDRED DAYS OF CODE

DAY 18

case 4:

```
    duckNumber();
```

```
    break;
```

case 5:

```
    printf("Exiting the program...\n");
```

```
    break;
```

default:

```
    printf("Invalid choice! Please enter a choice from 1 to 5.\n");
```

```
}
```

```
} while (choice != 5);
```

```
return 0;
```

```
}
```

6.TEST CASE TABLE:

TEST CASE	MENU CHOICE	INPUT	
1	1	145	145 is a Strong number
2	2	153	153 is an Armstrong number
3	3	18	18 is a Harshad number
4	4	1234	1234 is not a duck number
5	5	----	EXITING THE PROGRAM

COMPLIATION AND EXECUTION:

HUNDRED DAYS OF CODE

DAY 18

```
(kali@kali)-[~/day17]  
$ vi duck.c
```

```
(kali@kali)-[~/day17]  
$ gcc duck.c -o duck
```

```
(kali@kali)-[~/day17]  
$ ./duck
```

```
===== MENU =====
```

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit

```
=====
```

Enter your choice: 1

Enter a positive integer: 145
145 is a Strong number.

```
===== MENU =====
```

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit

```
=====
```

Enter your choice: 2
Enter a positive integer: 153
153 is an Armstrong number.

HUNDRED DAYS OF CODE

DAY 18

```
===== MENU =====
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit
=====
Enter your choice: 3
Enter a positive integer: 18
18 is a Harshad number.

===== MENU =====
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit
=====
Enter your choice: 4
Enter a positive integer: 1234
1234 is not a Duck number.
```

```
===== MENU =====
1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit
=====
Enter your choice: 5
Exiting the program...

(kali@kali)-[~/day17]
$
```